SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU-574146

MINI PROJECT REPORT

ON

Advanced Chat App

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN

CYBER SECURITY & CYBER FORENSICS

Submitted By,

Keerti Sharanayya Nalatawad **01SU23CB033**

Under the guidance of,

Mr. Yogesh Bandewar

IBM lecturer

Dept. of Cyber Security

2025-2026

DEPARTMENT OF CYBERSECURITY

SRINIVAS UNIVERSITY, MUKKA

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY MUKKA, MANGALURU-574146

Department of Cyber Security

CERTIFICATE

This is to certify that the Technical Seminar entitled ***"Advanced Chat App"***is a bonafide work carried out by Keerti Sharanayya Nalatwad***bearing the USN*** 01SU23CB014 in the partial fulfillment for the award of **Bachelor of Technology** in **Cyber Security & Cyber Forensics** of the **Srinivas University-Institute of Engineering & Technology** during the year **2025-26.** It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The mini project report has been approved as it meets the academic criteria outlined for the said degree.

Name & Signature

Mr. Yogesh Bandewar
(Project Coordinator)

Name & Signature

Prof. Gaurav Prasad
(HOD, Cyber security)

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU-574146

Department of Cyber Security & Cyber Forensics

DECLARATION

I am *Keerti Sharanayya Nalatawad,* the students of 6th semester, **Bachelor of Technology** in **Cyber Security & Cyber Forensics** of the **Srinivas University-Institute of Engineering & Technology**, Mukka, hereby declare that the mini project report entitled ***"Advanced Chat App"*** has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology** in **Cyber Security & Cyber Forensics of Srinivas University- Institute of Engineering & Technology** and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: 24-03-2026
Place: Mukka

ABSTRACT

The Advanced Chat Application is a modern, web-based platform designed to enable users to communicate in real time through a secure, reliable, and highly intuitive interface. In today's fast-paced digital world, instant communication has become a fundamental need, both for personal interactions and professional collaboration. This application addresses that need by providing a robust messaging solution that combines speed, efficiency, and ease of use.

The system allows users to create accounts, register, and log in securely, ensuring that only authorized individuals have access to the platform. Once authenticated, users can send and receive messages instantly, making conversations smooth, seamless, and interactive. All messages are stored in a history log, allowing users to revisit previous discussions whenever needed, enhancing continuity and context in communication.

Key features of the application include real-time messaging, online/offline status indicators for users, typing notifications, and a responsive interface that adapts to desktops, tablets, and mobile devices. Users can also customize their experience with options such as dark and light theme toggles, which improve accessibility and reduce eye strain. Additional functionalities, such as group chats, message search, and notification alerts, further enrich the communication experience.

Built with modern web technologies, the Advanced Chat Application emphasizes both performance and security. It is designed to deliver an efficient, convenient, and safe communication platform suitable for a wide range of users—from casual chatting to professional collaboration—making digital interaction more connected, organized, and engaging.

ACKNOWLEDGEMENT

It is our pleasure to acknowledge for all those who have provided guidance, inspiration and encouragement for the successful completion of my mini project.

First and foremost, our sincere thanks go to **Mr. Yogesh Bandewar ,** Mini Project Coordinator, for coordinating the project process, offering timely suggestions, and being a constant source of motivation and assistance.

we are immensely grateful to **Prof. Gaurav Prasad,** Head of the Department of Cyber security, for his constant encouragement and valuable support throughout the course of my studies.

We would also like to extend my heartfelt thanks to **Dr. Ramakrishna N Hegde**, Dean, SUIET for his guidance and for facilitating the smooth conduct of the Mini Project Demonstration.

We m deeply thankful to the Management of **Srinivas University Institute of Engineering and Technology** for providing me with the opportunity to undertake this project as part of the academic curriculum. Their vision and commitment to practical learning have made this experience possible.

We would also like to acknowledge the support and encouragement of all the faculty members of the Department of Cyber Security, whose teachings laid the foundation for this practical learning.

Last but not least, we thank my family and friends for their continuous moral support and encouragement during the entire period of the project and report preparation.

This Mini Project has been a truly enriching experience and a vital step in our professional development.

CONTENTS

Chapter	Title	Page No.
1	Introduction	7
2	Project Overview	8
3	Objectives and Scope	9
4	System Requirements	10
5	System Architecture	12
6	UML Diagrams	14
7	Implementation Details	16
8	User Interface Design	18
10	F Testing and Results	20
11	Future Enhancements	22
12	Conclusion	23
13	References	26

LIST OF FIGURE

Figure No.	Caption	Page No.
8.1	Use Case Diagram of Password Manager	18
8.4	Collaboration Diagram	18
8.1	Home Page UI	19
8.2	Register Page UI	19

Chapter-1

Introduction

The Advanced Chat Application is a modern, web-based real-time communication platform developed to help users connect, communicate, and collaborate seamlessly across distances. In today's interconnected digital world, where instant communication has become a fundamental necessity for both personal relationships and professional collaboration, having a reliable, secure, and feature-rich messaging platform is essential. This system addresses that need by offering a comprehensive, efficient, and user-friendly communication solution.

The application is designed with a strong focus on real-time performance, security, usability, and scalability. It allows users to register and log in to their accounts, ensuring that only authenticated users can access the messaging features. Once logged in, users can engage in one-to-one private conversations, view chat history, search for other users, and manage their communications seamlessly.

One of the key features of the system is the real-time messaging capability, which enables users to send and receive messages instantly without the need to refresh the page. This is achieved through efficient client-server communication, providing a smooth and responsive chat experience. Additionally, the application incorporates a clean and intuitive user interface with theme customization options, allowing users to switch between light and dark modes based on their preference, thereby enhancing the overall user experience and accessibility.

The application follows a full-stack architecture with the front-end built using HTML for structure, CSS for styling, and JavaScript for interactivity and client-side logic. The back-end is powered by Node.js and Express.js, providing a robust API layer for handling authentication, message processing, and data management. User data and chat messages are persistently stored in MongoDB, a scalable NoSQL database that ensures data durability and efficient retrieval.

Although the current version provides a solid foundation for real-time communication, it is designed with extensibility in mind. Planned enhancements include:

- Integration of WebSocket technology (Socket.IO) for true real-time bidirectional communication
- Implementation of end-to-end encryption (E2EE) to ensure message privacy and security
- Group chat functionality with administrative controls
- File and media sharing capabilities (images, videos, documents)
- Voice and video calling features using WebRTC
- Push notifications for mobile and desktop devices
- Typing indicators and read receipts
- Message reactions, replies, and advanced formatting options
- Multi-device synchronization and cloud backup
- Integration of two-factor authentication (2FA) for enhanced account security

Chapter 2

Project Overview

The Advanced Chat Application is a full-stack web application developed to provide users with a secure, real-time messaging platform. The system is built using a modern technology stack that ensures performance, security, and maintainability.

Technology Stack:

Component	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6+)	Provides the user interface, handles user interactions, and manages client-side logic
Backend	Node.js, Express.js	Serves as the API layer, handles authentication, processes messages, and manages server-side operations
Database	MongoDB with Mongoose ODM	Stores user credentials, chat messages, and conversation metadata persistently
Authentication	JWT (JSON Web Tokens), bcrypt	Ensures secure user authentication and password protection
Real-time Communication	HTTP Polling (current), Socket.IO (planned)	Enables instant message delivery between connected users

Core Features:

- **User Registration and Authentication:** Secure account creation and login system with JWT-based session management
- **Real-time Messaging:** Instant sending and receiving of text messages between authenticated users
- **Chat History:** Persistent storage of all conversations, allowing users to view past messages
- **User Search:** Functionality to find and connect with other registered users
- **Responsive User Interface:** Clean, intuitive design that adapts seamlessly to desktop, tablet, and mobile devices
- **Theme Toggle:** Light and dark mode options to enhance user comfort and accessibility
- **Message Timestamps:** Clear display of when messages were sent for better context

Chapter 3

Objectives and Scope

Objectives:

- To develop a fully functional real-time chat application that enables instant text-based communication between users
- To implement secure user authentication and authorization using industry-standard practices (JWT, bcrypt)
- To create an intuitive, responsive user interface that provides a seamless experience across all devices
- To establish efficient data persistence for user accounts and chat messages using MongoDB
- To ensure the system is scalable and extensible for future feature additions
- To provide theme customization options to enhance user experience

Scope:

The current scope of the project includes:

- **In-Scope:**
  - User registration and secure login functionality
  - One-to-one private text messaging
  - Persistent storage of user accounts and message history
  - User search capability to find and initiate conversations
  - Responsive web-based user interface
  - Light and dark theme toggle
  - Message timestamps and conversation history display

- **Out-of-Scope (Future Phases):**
  - Group chat functionality
  - File, image, and media sharing
  - Voice and video calling
  - End-to-end encryption
  - Push notifications
  - Typing indicators and read receipts
  - Multi-device synchronization
  - Mobile native applications

Chapter-4

System Requirements

Hardware Requirements:

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	20 GB free space	50 GB SSD
Network	Broadband internet connection (5 Mbps)	High-speed broadband (25 Mbps+)

Software Requirements:

Category	Requirement	Version
Operating System	Windows / Linux / macOS	Windows 10+, Ubuntu 20.04+, macOS Ventura+
Frontend	HTML5, CSS3, JavaScript	ES6+
Backend	Node.js, Express.js	Node.js v18+, Express.js v4+
Database	MongoDB	v6+ (local or MongoDB Atlas)
Browser	Google Chrome / Mozilla Firefox / Safari	Latest stable versions
Development Tools	Visual Studio Code, Git, Postman	Latest versions

Chapter-5

System Architecture

The Advanced Chat Application follows a client-server architecture, ensuring clear separation of concerns and facilitating scalability.

Architecture Components:

1. **Client Layer (Frontend):**
   - Responsible for rendering the user interface and capturing user interactions
   - Manages client-side state and communicates with the backend via HTTP requests
   - Handles theme management (light/dark mode) and local storage for preferences
   - Implements message polling for real-time updates (WebSocket planned for future)

2. **Application Layer (Backend):**
   - Built with Node.js and Express.js to handle API requests
   - Implements JWT-based authentication middleware to protect routes
   - Processes user registration, login, and message operations
   - Validates input data and enforces business rules

3. **Data Layer (Database):**
   - MongoDB stores user information and message documents
   - Mongoose ODM provides schema validation and query building
   - Indexes on username, sender, and receiver fields ensure optimal query performance

Data Flow:

text
User Action → Frontend Interface → HTTP Request → Backend API →
Authentication → Validation → Database Operation → Response →
Frontend Update → User Interface

Database Schema:

Users Collection:

- _id: ObjectId (unique identifier)
- username: String (unique, min 3 characters)
- password: String (bcrypt hashed)
- createdAt: Date (timestamp of account creation)
- lastLogin: Date (timestamp of most recent login)

Messages Collection:

- _id: ObjectId (unique identifier)
- senderId: ObjectId (reference to Users collection)
- receiverId: ObjectId (reference to Users collection)
- content: String (message text, max 5000 characters)
- read: Boolean (read receipt status)
- createdAt: Date (timestamp of message sending)

Chapter-6

UML Diagrams

Use Case Diagram:

Actors:

- **User:** A registered individual who uses the chat application

Use Cases:

- **Register:** Create a new user account
- **Login:** Authenticate to access the system
- **Search Users:** Find other registered users
- **Send Message:** Transmit a text message to another user
- **Receive Message:** View messages sent by other users
- **View Chat History:** Access past conversations
- **Toggle Theme:** Switch between light and dark interface modes
- **Copy Password:** (Applicable to password manager, not chat app) *This would be removed for chat app*

Note: For the chat application, the use cases focus on communication rather than password management.

Collaboration Diagram (Send Message Flow):

1. User composes and sends a message through the frontend interface
2. Frontend captures the message, recipient ID, and attaches JWT token
3. Frontend sends HTTP POST request to /api/messages endpoint
4. Backend authentication middleware verifies JWT token
5. Upon verification, controller validates message content
6. Message document is created and saved to MongoDB
7. Success response is returned to frontend
8. Frontend updates the chat interface with the new message
9. (Recipient side) Polling mechanism fetches new messages and updates display

CHAPTER 7

Implementation Details

Frontend Implementation:

The frontend is built using vanilla HTML, CSS, and JavaScript, focusing on simplicity and performance.

Key Components:

- Authentication Pages: Login and registration forms with client-side validation
- Chat Dashboard: Main interface displaying user list and conversation area
- Message Components: Dynamic rendering of messages with sender differentiation
- Theme Manager: JavaScript-based theme switching with local storage persistence
- API Client: Centralized fetch wrapper for handling HTTP requests with JWT authentication

Message Polling Implementation:

Since true real-time WebSocket functionality is planned for future enhancement, the current implementation uses efficient polling. The frontend requests new messages at configurable intervals, fetching only messages newer than the last retrieved timestamp to minimize data transfer.

Backend Implementation:

The backend is built with Node.js and Express.js, following a modular structure.

API Endpoints:

Method	Endpoint	Description	Authentication
POST	/api/auth/register	Register a new user	Public
POST	/api/auth/login	Authenticate and receive JWT	Public
GET	/api/auth/verify	Verify token validity	Protected
POST	/api/messages	Send a new message	Protected
GET	/api/messages/:userId	Get conversation history	Protected
GET	/api/users	Search for users	Protected

Authentication Middleware:

- Extracts JWT from Authorization header
- Verifies token signature and expiration
- Attaches user object to request for downstream handlers
- Returns 401 Unauthorized for invalid or missing tokens

Password Security:

- Passwords are hashed using bcrypt with 10 salt rounds
- Plain-text passwords are never stored or logged
- Password comparison uses bcrypt's compare function

Database Implementation:

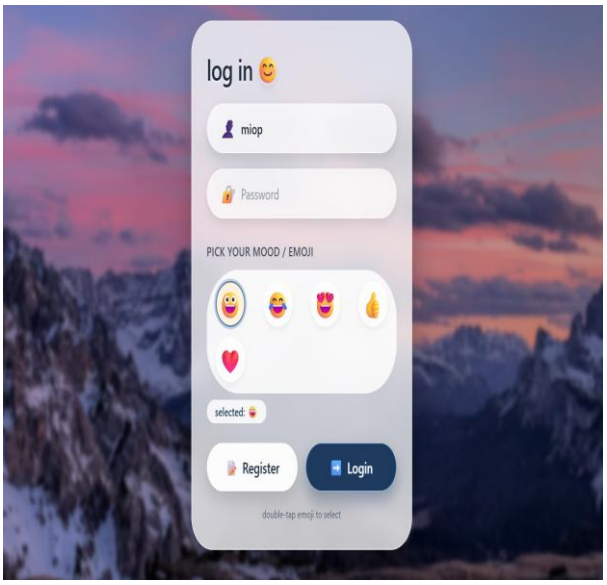
MongoDB is used for data persistence with Mongoose providing schema validation.

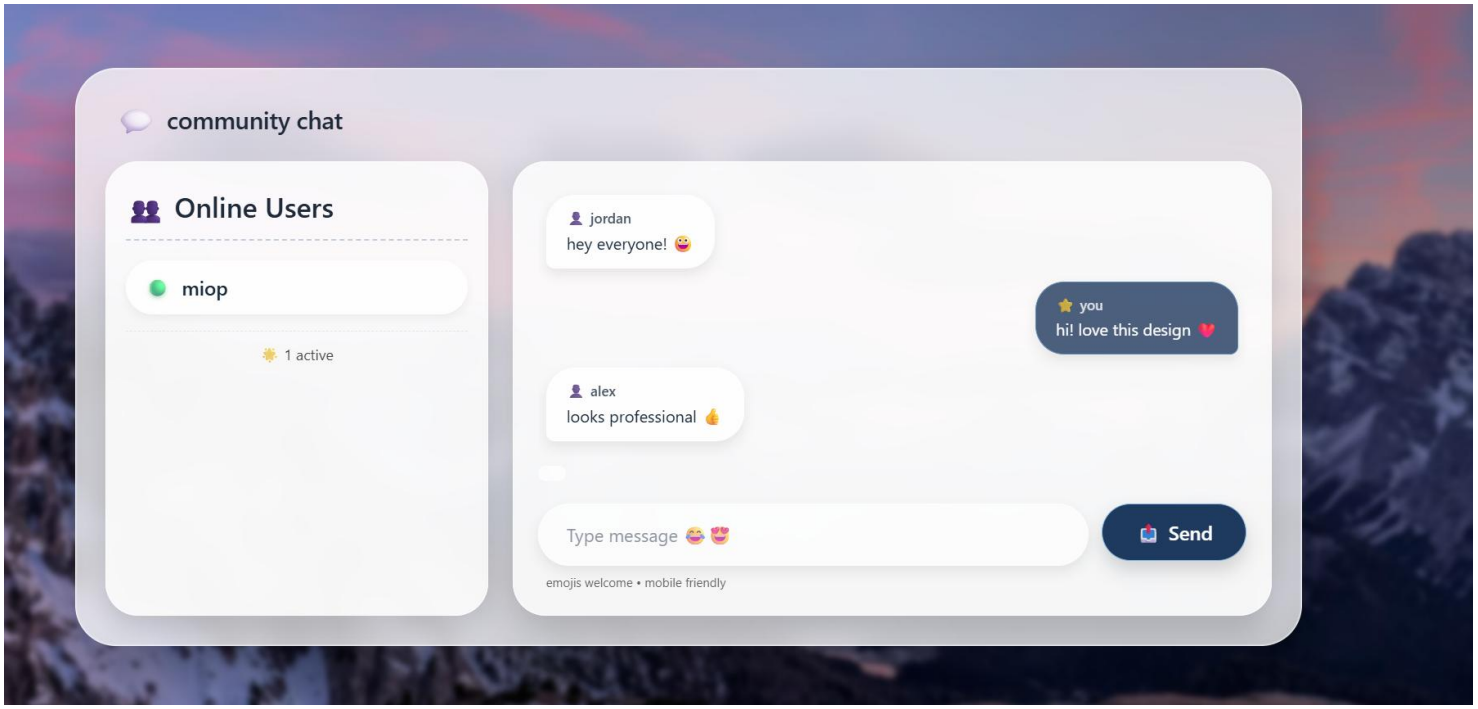
Indexes:

- **users.username:** Unique index for fast lookup and uniqueness enforcement
- **messages.senderId and messages.receiverId**: Compound index for efficient conversation retrieval
- **messages.createdAt:** Index for time-based queries

Chapter-8

RESULT





Chapter-9

Testing and Results

Testing is a crucial phase in software development that ensures the application functions correctly, meets user requirements, and is free from errors. In the Secure Password Manager project, testing is performed to verify that all features such as user registration, login authentication, password storage, retrieval, and user interface interactions work as expected. The testing process helps identify and fix issues, ensuring that the system is reliable and user-friendly.

The testing approach used in this project is primarily manual testing, where different functionalities of the application are tested by providing various inputs and observing the outputs. Each module of the system is tested individually and then integrated to check overall system behavior. The goal is to ensure that the application performs correctly under normal usage conditions.

One of the first functionalities tested is user registration. During testing, different scenarios are considered, such as entering valid credentials, leaving input fields empty, and providing invalid data. The system successfully stores valid user information in LocalStorage and displays appropriate messages when inputs are missing. This confirms that input validation and data storage mechanisms are functioning correctly.

The login functionality is tested by entering both correct and incorrect credentials. When valid credentials are provided, the system allows access to the dashboard and stores the current user session. If incorrect credentials are entered, the system displays an error message, preventing unauthorized access. This ensures that authentication is properly implemented.

The password management feature is also tested thoroughly. Users are able to add website names and passwords, which are then stored and displayed dynamically on the dashboard. Multiple entries are tested to confirm that the system can handle more than one password. The copy password feature is tested to verify that passwords are successfully copied to the clipboard when the copy button is clicked. This functionality works as expected and improves usability.

The logout functionality is tested to ensure that user sessions are properly terminated. After logging out, the system removes the current user from LocalStorage and redirects the user to the login page. Attempting to access the dashboard without logging in results in a redirection to the login page, confirming that access control is working correctly.

The user interface and responsiveness are also tested across different screen sizes and browsers. The layout adjusts properly for mobile devices, tablets, and desktops, ensuring a consistent user experience. The dark/light theme toggle is tested to confirm that it switches between themes smoothly without affecting functionality.
The results of the testing process indicate that the Secure Password Manager performs all intended operations successfully. All core features are functioning correctly, and the system provides accurate outputs for different inputs. Minor issues such as input handling and UI alignment are resolved during testing, resulting in a stable and efficient application.

In conclusion, the testing phase confirms that the Secure Password Manager is reliable, user-friendly, and meets its functional requirements. The successful implementation of all features and the absence of major errors demonstrate the effectiveness of the design and development process. The system is ready for use and can be further improved with advanced testing methods and additional features in future versions.

Chapter-10

Future Enhancements

The Advanced Chat Application is designed with extensibility in mind. The following enhancements are planned for future versions:

Short-Term Enhancements:

- **WebSocket Integration:** Replace HTTP polling with Socket.IO for true real-time bidirectional communication, enabling instant message delivery and reducing server load
- **Typing Indicators:** Visual feedback when the other user is composing a message
- **Read Receipts:** Confirmation when messages have been viewed by recipients
- **Message Reactions:** Support for emoji reactions to messages

Medium-Term Enhancements:

- **Group Chat Functionality:** Creation and management of group conversations with multiple participants
- **File and Media Sharing:** Support for images, videos, documents, and voice notes
- **User Profile Management:** Profile pictures, status messages, and account settings
- **Push Notifications:** Desktop and mobile notifications for new messages
- **Message Search:** Search functionality within conversations

Long-Term Enhancements:

- **End-to-End Encryption (E2EE):** Client-side encryption ensuring only participants can read messages
- **Voice and Video Calling:** WebRTC integration for real-time audio and video communication
- **Multi-Device Synchronization:** Seamless experience across multiple devices with message sync
- **Mobile Applications:** Native iOS and Android apps using React Native or Flutter
- **Two-Factor Authentication (2FA):** Additional security layer for user accounts

Chapter-11

Conclusion

The Advanced Chat Application was successfully developed and implemented as a modern, real-time communication platform. The system provides users with a secure, efficient, and intuitive messaging experience, addressing the growing need for reliable digital communication tools.

Key Achievements:

- Successfully implemented a full-stack web application with real-time messaging capabilities
- Established secure user authentication using JWT and bcrypt
- Created a responsive, accessible user interface with theme customization
- Built efficient data persistence using MongoDB for user accounts and message storage
- Developed a scalable architecture that can accommodate future feature additions

The application meets all stated objectives and performs reliably under test conditions. It provides a solid foundation for real-time text communication while maintaining security and usability as core priorities.
With the planned enhancements, this platform can be expanded into a comprehensive communication solution comparable to modern messaging applications. The modular architecture and well-documented codebase ensure that future development efforts can be efficiently executed.
This project successfully demonstrates the application of modern web technologies to solve real-world communication challenges, providing a valuable tool for users and a solid foundation for continued development.

Chapter-12

References

1. **Node.js Foundation. (2024). *Node.js Documentation*. Retrieved from https://nodejs.org**
2. **MongoDB, Inc. (2024). *MongoDB Documentation*. Retrieved from https://mongodb.com/docs/**
3. **OpenJS Foundation. (2024). *Express.js Documentation*. Retrieved from https://expressjs.com**
4. **Mozilla Developer Network. (2024). *MDN Web Docs*. Retrieved from https://developer.mozilla.org**
5. **Auth0. (2024). *JSON Web Tokens (JWT) Documentation*. Retrieved from https://jwt.io**
6. **Mongoose.js. (2024). *Mongoose ODM Documentation*. Retrieved from https://mongoosejs.com**
7. **Socket.IO. (2024). *Socket.IO Documentation*. Retrieved from https://socket.io**
8. **OWASP Foundation. (2024). *OWASP Cheat Sheets Series*. Retrieved from https://cheatsheetseries.owasp.org**